# Software Re-Engineering

## Lecture 2

Static code analysis

**Instructor:**

Dr. Natalia Chaudhry,

*Assistant Professor, Software Engineering Department, PU.*

SE Fall 2020

# Agenda

- Deciding where to focus your refactoring efforts
- Measuring the quality of software
- Inspecting your codebase using **FindBugs, PMD,** and **Checkstyle**
- Using Jenkins for continuous inspection



# Static code analysis

Test the quality of code by analyzing source code

# Refactoring

anything you can think of that makes the code cleaner and more readable.
This process is known as **exploratory refactoring**

## BENEFITS OF EXPLORATORY REFACTORING

**increases your understanding of the code**

- This will make it a lot easier to make changes in the future, whether they're bug fixes, new features, or more refactorings, without any fear of unexpected side effects.

**exploratory refactoring improves the readability of the code**

# Gathering useful data about your software

**Bugs and coding standard violations**
Static analysis tools can analyze a codebase and detect possible bugs or poorly written code. Static analysis involves looking through the code (either the human-readable source code or the machine-readable compiled code) and flagging any pieces of code that match a predefined set of patterns or rules.
For Java code, the big three tools are **FindBugs, PMD,** and **Checkstyle**

**Performance**

One goal of your refactoring may be to improve the performance of a legacy system. If that's the case, you'll need to measure that performance

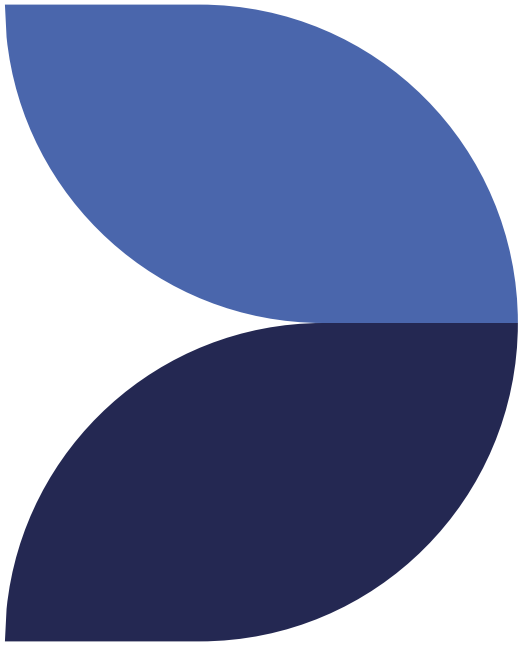
**Error counts**

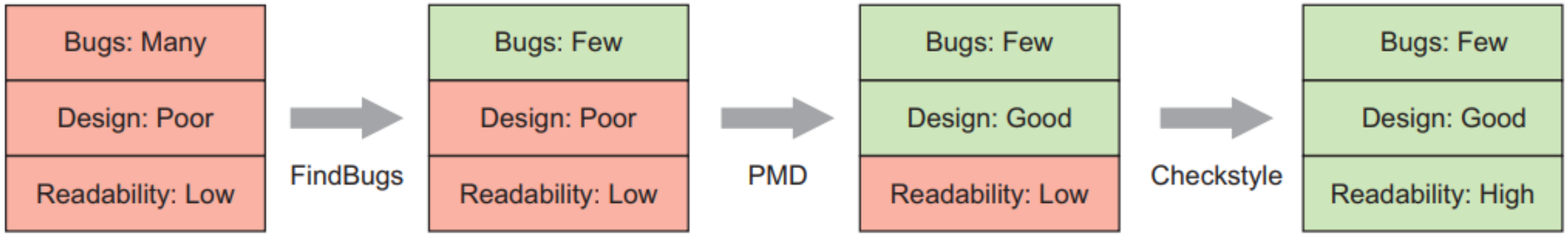
Measuring performance is all well and good, but it doesn't matter how fast your code runs if it's not doing its job correctly (giving users the results they expect and not throwing any errors). A count of the number of errors happening in production is a simple but useful indicator of the quality of your software, as seen from the end-user's perspective.

**Commonly used files**

Knowing which files in your project are edited most often can be very useful when choosing your next refactoring target. If one particular class is edited by developers very often, it’s an ideal target for refactoring.

# Inspecting your codebase using FindBugs, PMD, and Checkstyle






**Improving code using static analysis tools**

- FindBugs is a free, open source tool
- It works by analyzing the bytecode generated by the Java compiler and searching for suspicious patterns.
- For each instance it finds, it assigns a confidence rating, indicating how likely it believes the code to be buggy.
- Each pattern in the FindBugs database is also assigned a scariness ranking, indicating how bad it would be if that kind of bug existed in your code.
- For example, a NullPointerException usually has a much more serious effect on the running of your program

- *FindBugs can be run in a number of ways, but the easiest way is to use an IDE plugin.*
- *Plugins are available for Eclipse, IntelliJ IDEA, NetBeans, and probably many other IDEs.*

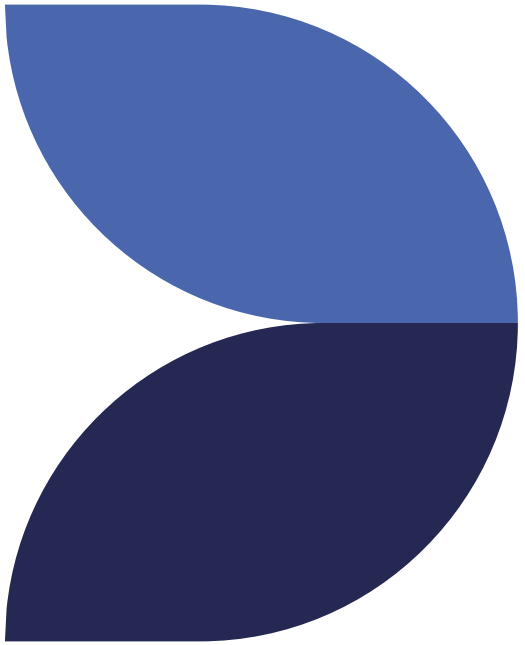
# Contd..

**DON’T FORGET TO COMPILE**

FindBugs runs on compiled bytecode, whereas what you see in your IDE is the source code. If the source code and bytecode aren’t in sync, the result of a FindBugs analysis can be quite confusing. Make sure you’ve compiled the code before running FindBugs

**Download eclipse:**
https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2023-09/R/eclipse-inst-jre-win64.exe

# FindBugs



- **FindBugs** is an open source tool used to perform static analysis on Java code.
- Findbugs provides early feedback about potential errors in the code.
- This helps the developer to access these problems early in the development phase.

http://findbugs.cs.umd.edu/eclipse

https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2023-09/R/eclipse-inst-jre-win64.exe

## Main attributes of the FindBugs – namely **rank** and **confidence**.

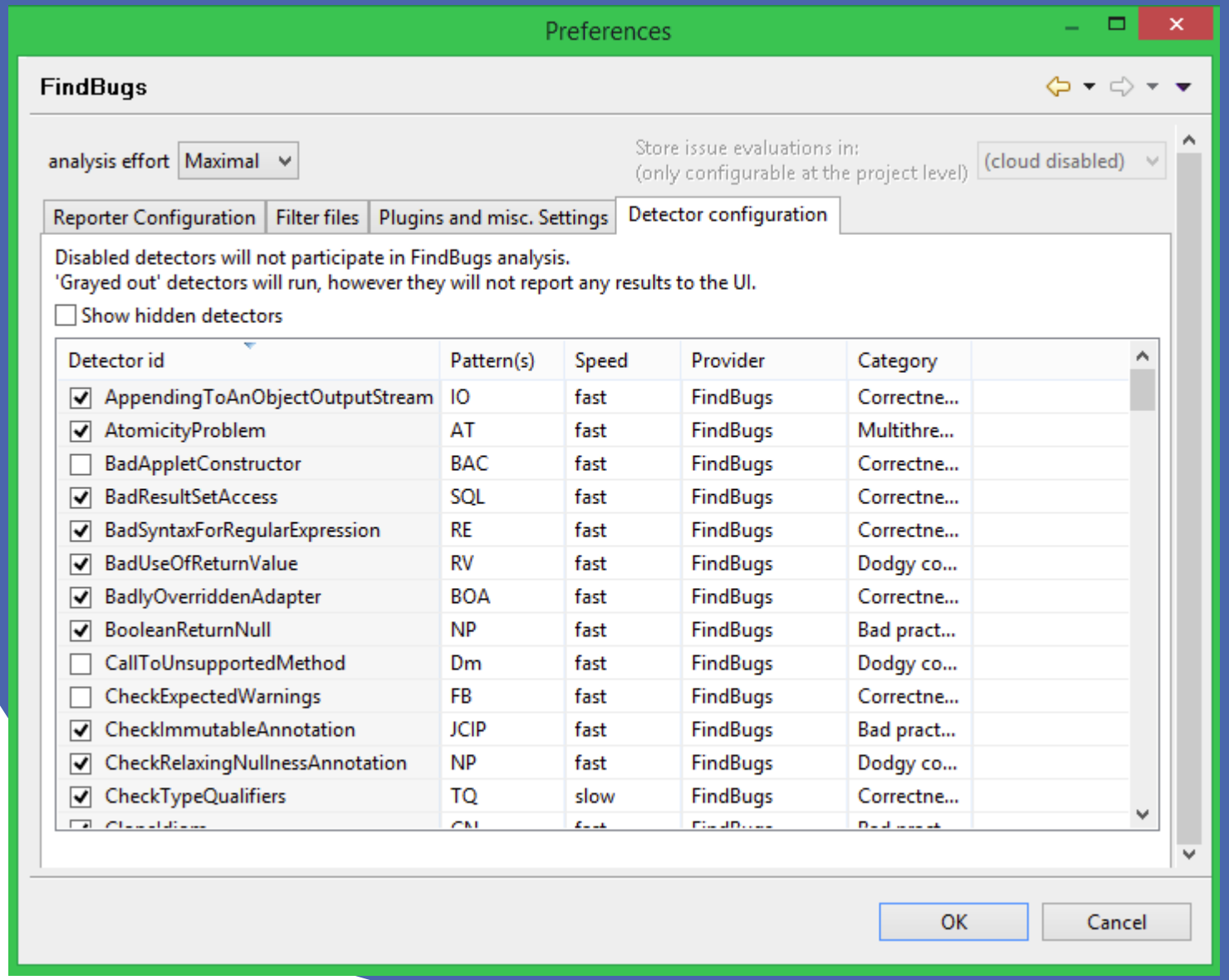
Bugs are given a **rank** 1-20, and grouped into the categories
1.Scariest (rank 1-4)
2.Scary (rank 5-9)
3.Troubling (rank 10-14)
4.Of concern (rank 15-20)

While the bug rank describes severity, the confidence factor reflects the likelihood of these bugs to be flagged as real ones. The **confidence** was originally called priority, but it was renamed in the new version.

**FindBugs** divide defects in many categories:

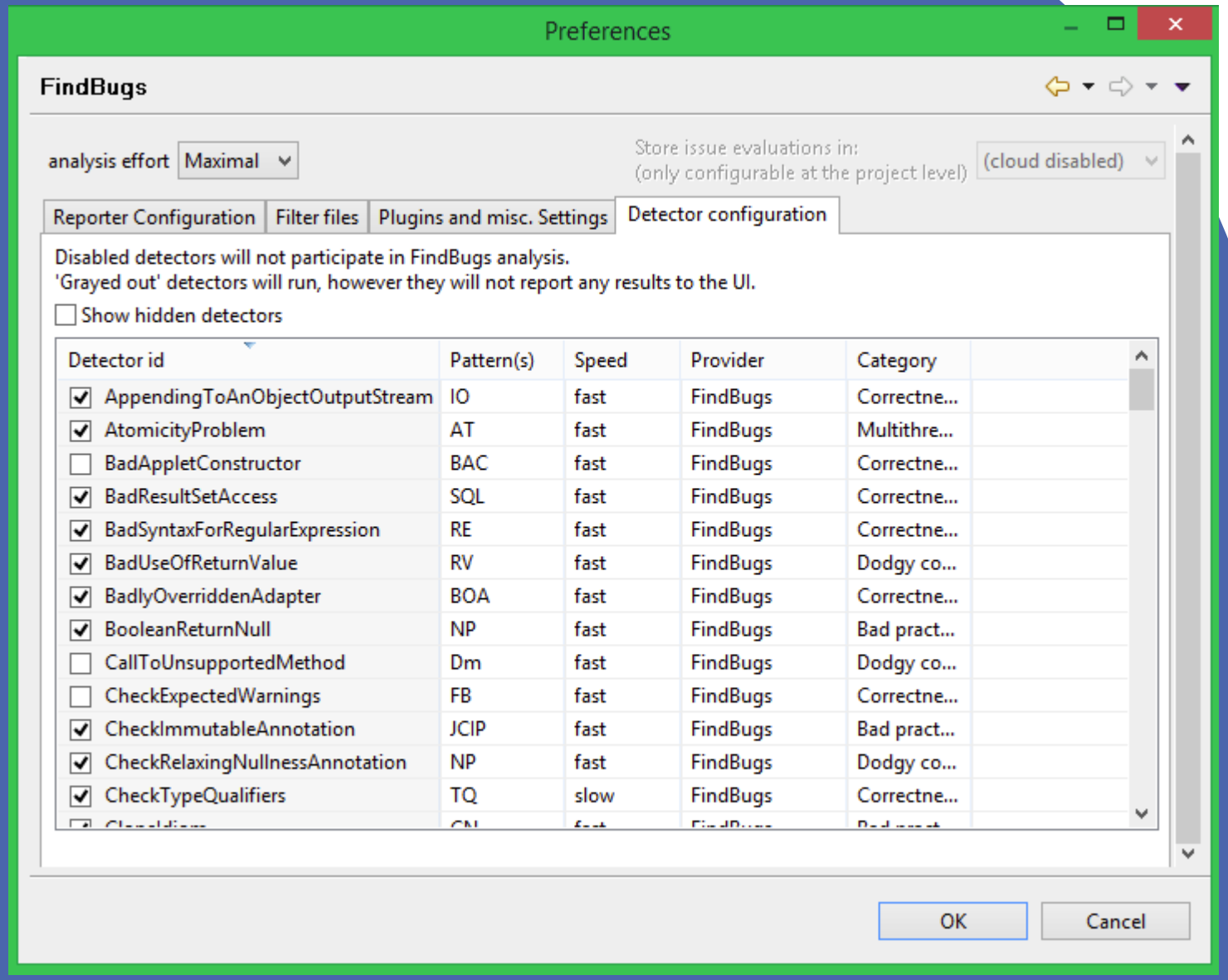
- **Correctness** – gathers general bugs, e.g. infinite loops, inappropriate use of equals(), etc
- **Bad practice**, e.g. exceptions handling, opened streams, Strings comparison, etc
- **Performance**, e.g. idle objects
- **Multithreaded correctness** – gathers synchronization inconsistencies and various problems in a multi-threaded environment
- **Internationalization** – gathers problems related to encoding and application's internationalization
- **Malicious code vulnerability** – gathers vulnerabilities in code, e.g. code snippets that can be exploited by potential attackers
- **Security** – gathers security holes related to specific protocols or SQL injections
- **Dodgy** – gathers code smells, e.g. useless comparisons, null checks, unused variables, etc

Under the **Detector configuration** tab, you can check the rules you're supposed to respect in your project:




The **speed attribute** reflects how costly the analysis will be. The fastest the detector, the smallest the resources consumed to perform it.

Under the **Filter files panel**, you can create custom file filters, in order to include/exclude parts of the code-base. This feature is useful – for example – when you want to prevent “unmanaged” or “trash” code, defects to pop up in the reports, or may exclude all classes from the test package for instance.

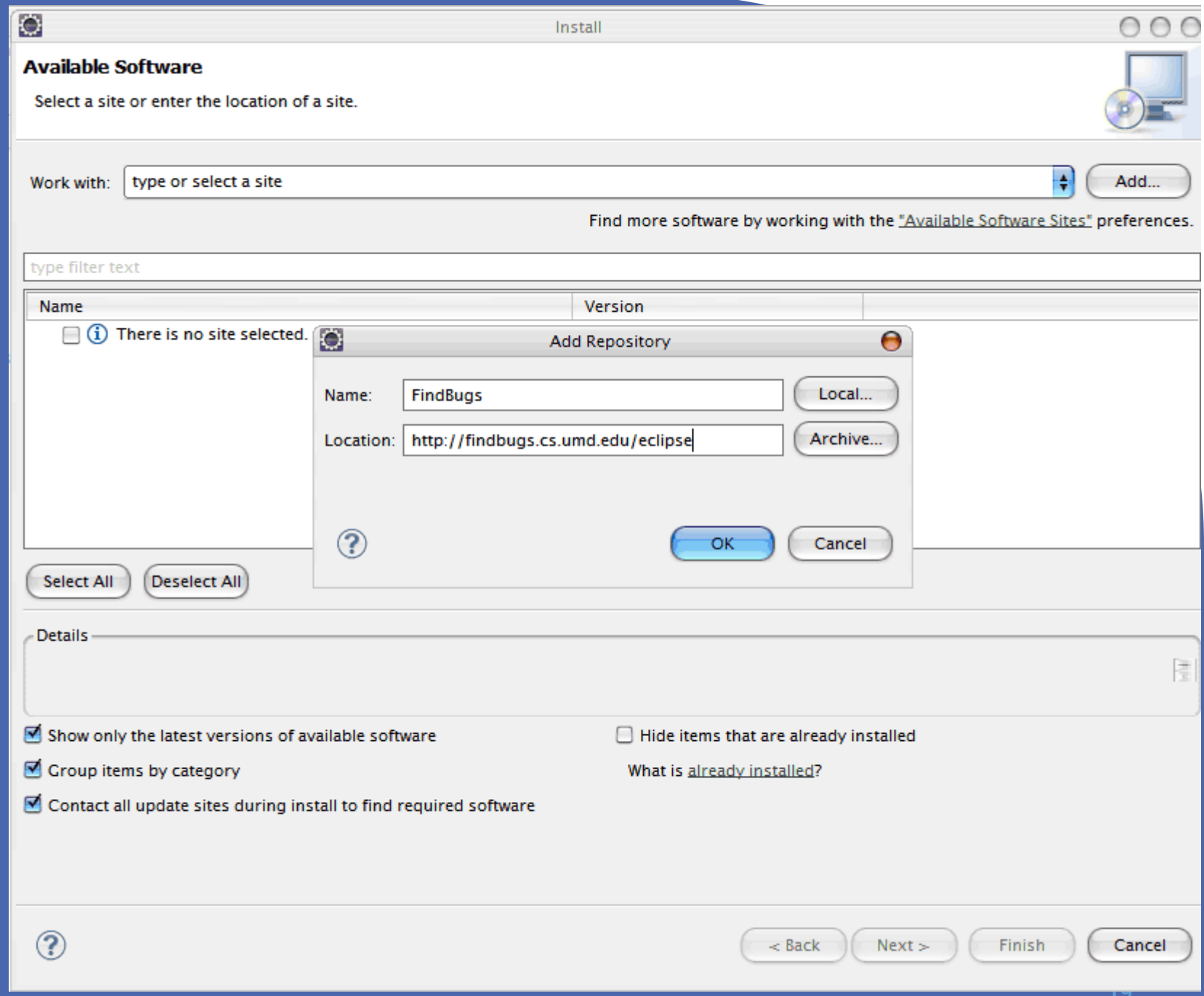


1. Installing the FindBugs Plugin in Eclipse.
2. Configuring the FindBugs properties in Eclipse
3. Analyzing code with FindBugs within Eclipse

**FindBugs Installation**

For Installing FindBugs Plugin from eclipse, select Help->Install New Software Then click add, then provide Name and Location according to the following screenshot

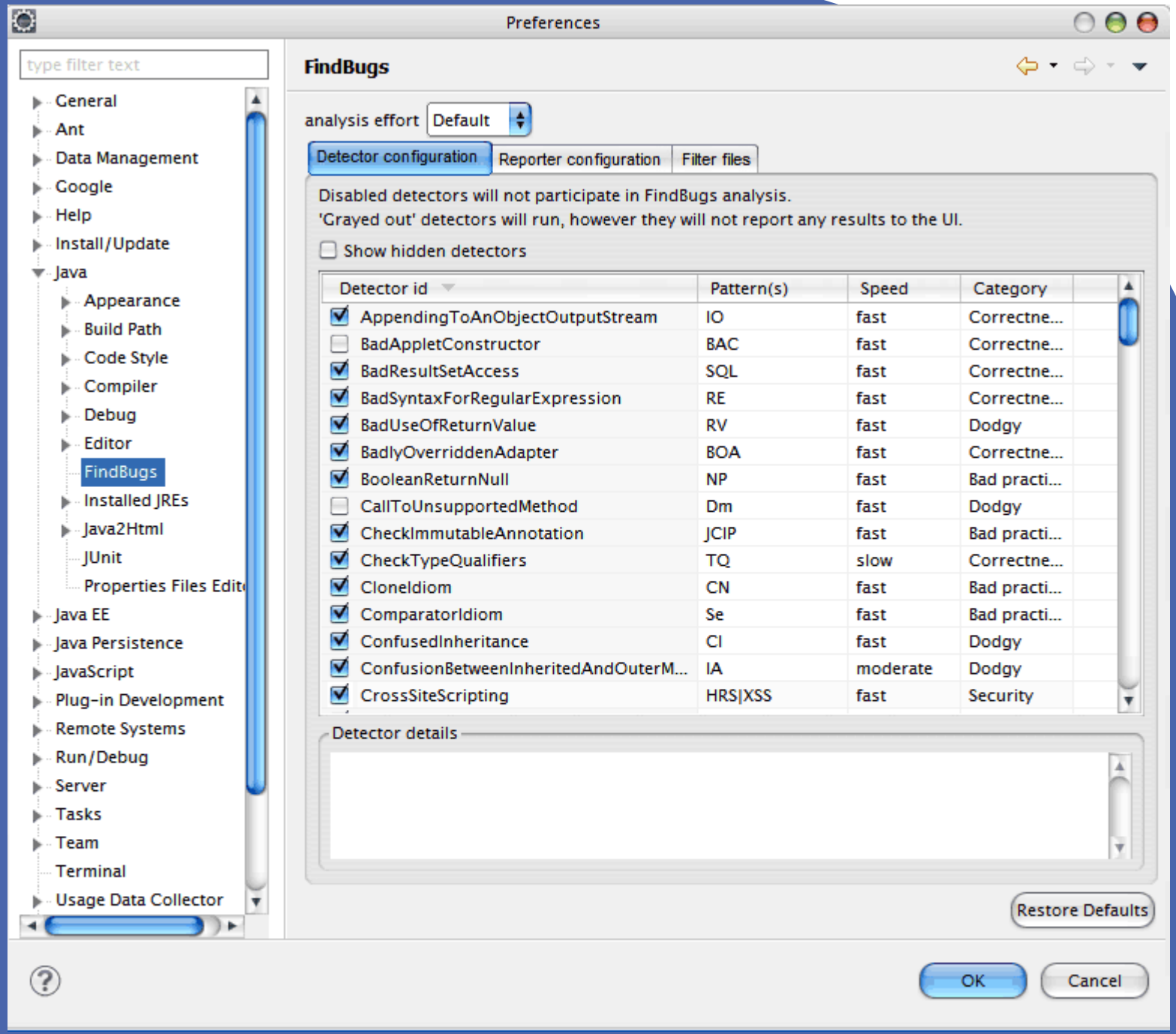
Then select the FindBugs version click next and accept the license details, it will install the FindBugs



## FindBugs Configuration

After install, you can configure FindBugs from eclipse, Window->Preferences->Java->FindBugs

Here you can apply FindBugs filter, in order to utilizing different Findbugs bug patterns



## Using FindBugs Plugin

Finding bugs using FindBugs, you need to create a java project and add the Java code to it for testing

```
public class FindBugsTest {
  public static void main(String args[]){
    System.out.println(new Integer(10));
  }
}
```

**FindBugs is analyzing the byte code, so you need to build the project before running FindBugs.**

right-click on a project and select *Properties* from the popup menu.
Select **FindBugs**

```java
package demo1;


public class demo1 {
public static void main (String args[])
{
    int i=0;
    String test=null;
    test.length();
    System.out.println("demo 1");
}


}
```

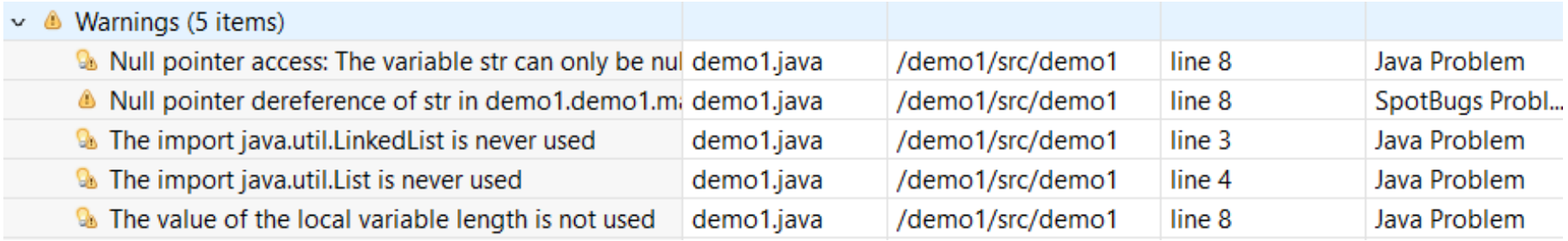
```
public class SampleClass {

    public void divideByZero() {
        int numerator = 10;
        int denominator = 0;
        int result = numerator / denominator;
        System.out.println("Result: " + result);
    }

    public static void main(String[] args) {
        SampleClass sample = new SampleClass();
        sample.divideByZero();
    }
}
```

## 1. Null Pointer Dereference:

```
public class NullPointerExample {
    public static void main(String[] args) {
        String str = null;
        int length = str.length(); // This will trigger a null
pointer exception
    }
}
```



| Warnings (5 items) | | | | |
|---|---|---|---|---|
| Null pointer access: The variable str can only be nul | demo1.java | /demo1/src/demo1 | line 8 | Java Problem |
| Null pointer dereference of str in demo1.demo1.m | demo1.java | /demo1/src/demo1 | line 8 | SpotBugs Probl... |
| The import java.util.LinkedList is never used | demo1.java | /demo1/src/demo1 | line 3 | Java Problem |
| The import java.util.List is never used | demo1.java | /demo1/src/demo1 | line 4 | Java Problem |
| The value of the local variable length is not used | demo1.java | /demo1/src/demo1 | line 8 | Java Problem |

## 2. Unused Fields or Variables:

```
public class UnusedVariableExample {
    private int unusedField;

    public void unusedMethod() {
        int unusedVar = 42;
    }
}
```

**3. Resource Leak:**

```
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;

public class ResourceLeakExample {
    public static void main(String[] args) throws IOException
{
        InputStream inputStream = new
FileInputStream("file.txt");
        // Missing close() method, can lead to resource leak
    }
}
```

FindBugs can detect security vulnerabilities in Java code. One example of a security-related bug pattern that FindBugs can identify is the "SQL Injection" vulnerability.

SQL injection is a common security issue where an attacker can manipulate input to execute arbitrary SQL queries against a database, potentially gaining unauthorized access or causing data breaches.

FindBugs can detect SQL injection vulnerabilities by analyzing code for patterns where user-provided input is concatenated directly into SQL queries without proper validation or sanitization. Here's an example of Java code that contains a SQL injection vulnerability:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SQLInjectionExample {
    public void updateUserPassword(Connection connection,
String username, String newPassword) {
        String query = "UPDATE users SET password='" +
newPassword + "' WHERE username='" + username + "'";

        try {
            PreparedStatement preparedStatement =
connection.prepareStatement(query);
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            // Handle the exception
        }
    }
}
```

In this code, user-provided input (username and newPassword) is directly concatenated into the SQL query string. An attacker can manipulate the input to execute malicious SQL queries, compromising the security of the application.

When you run FindBugs on code like this, it can identify the SQL injection vulnerability and report it as a security-related bug pattern, prompting developers to address the issue by using prepared statements or other secure database access methods to prevent SQL injection attacks.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SQLInjectionExample {
    public void updateUserPassword(Connection connection,
String username, String newPassword) {
        String query = "UPDATE users SET password='" +
newPassword + "' WHERE username='" + username + "'";

        try {
            PreparedStatement preparedStatement =
connection.prepareStatement(query);
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            // Handle the exception
        }
    }
}
```

Here's an example of a multithreaded correctness issue in Java code that FindBugs can detect:

Consider a scenario where two threads are simultaneously trying to increment a shared counter without proper synchronization:

```
public class MultithreadedCounter {
    private static int counter = 0;

    public static void main(String[] args) {
        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter++;
            }
        });

        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter++;
            }
        });

        thread1.start();
        thread2.start();

        try {
            thread1.join();
            thread2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Counter: " + counter);
    }
}
```

In this code, two threads (thread1 and thread2) are concurrently incrementing the counter variable without proper synchronization. This can lead to a race condition where the threads may read and update the counter simultaneously, resulting in unpredictable and incorrect values.

When you run FindBugs on code like this, it can detect the following multithreaded correctness issue:

Bug Pattern: "IS2_INCONSISTENT_SYNC" (Inconsistent synchronization)

Description: This bug pattern indicates that there is inconsistent synchronization in the code, meaning that the counter variable is accessed and modified by multiple threads without proper synchronization mechanisms like synchronized blocks or the use of AtomicInteger.

Recommendation: FindBugs recommends adding proper synchronization to ensure that concurrent access to the counter variable is safe and that race conditions are avoided.

By identifying such multithreaded correctness issues, FindBugs helps developers ensure the thread safety of their code and prevent unpredictable behavior in multithreaded applications.

## Handling false positives

FindBugs is a powerful tool, but there are limits to the capabilities of static analysis. Occasionally FindBugs may produce a false positive, flagging a piece of code as a bug even though you, a human, can see that it's not.

USING ANNOTATIONS Luckily, the FindBugs developers prepared for this eventuality and provided us with a way to mark a specific piece of code as "not a bug." You can use an annotation to tell FindBugs not to check a certain field or method against one or more rules.

```
@SuppressFBWarnings(
value = "UUF_UNUSED_FIELD",
justification = "This field is accessed using reflection")
private String name;
```

- The annotation has two fields: **value** and **justification**.
- You use the **value** field to tell FindBugs which bug patterns you want to suppress.
- The **justification** field is a comment to remind yourself and other developers why you've suppressed the warning

# PMD and Checkstyle



- Another popular static analysis tool for Java is PMD.
- PMD  stands for Programming Mistake Detector
- Unlike FindBugs, PMD works by analyzing Java source code rather than the compiled bytecode.
- This means it can search for some categories of problems that FindBugs can’t. For example, PMD can check for readability issues, such as inconsistent use of braces, or code cleanliness issues such as duplicate import statements
- Some of PMD’s most useful rules relate to **code design** and **complexity**. For example, it has rules for detecting excessive coupling between objects, or classes with high cyclomatic complexity. sd

CYCLOMATIC COMPLEXITY
Cyclomatic complexity is a count of the number of different paths your program can take inside a given method. Usually it’s defined as the sum of the number of if statements, loops, case statements, and catch statements in the method, plus 1 for the method itself. In general, the higher the cyclomatic complexity, the more difficult a method is to read and maintain.

## PMD IS NOISY BY DEFAULT

- When run with the default settings, PMD can be very noisy. For example, PMD wants you to add a final modifier to method arguments whenever possible, which can result in thousands of warnings for a typical codebase.
- There is a need to tune your PMD ruleset to disable some of the more noisy rules.

## SUPPRESSING WARNINGS

Just like FindBugs, PMD allows you to suppress warnings at the level of individual fields or methods using Java annotations. PMD uses the standard **@java.lang.SuppressWarnings annotation**, so there's no need to add any dependencies to your project. You can also disable PMD for a specific line of code by adding a NOPMD comment. The following listing shows the various ways of suppressing PMD warnings.

```
@SuppressWarnings("PMD")
public void suppressWarningsInThisMethod() {
    ...
}

@SuppressWarnings("PMD.InefficientStringBuffering")
public void suppressASpecificWarningInThisMethod() {
    ...
}

public void suppressWarningsOnOneLine() {
    int x = 1;
    int y = x + 1; //NOPMD
    ...
}
```

Suppresses all PMD warnings for the method

Suppresses a specific PMD warning

Suppresses all PMD warnings related to a specific line

**Install pmd using link:**

https://sourceforge.net/projects/pmd/files/pmd-eclipse/update-site/

(install new software)

Or use eclipse marketplace to install

After install, you can configure **PMD** from eclipse, **Window->Preferences->PMD**

Here you can apply PMD Rules, in order to utilizing different rules on PMD bug patterns, you can also configure different reports here

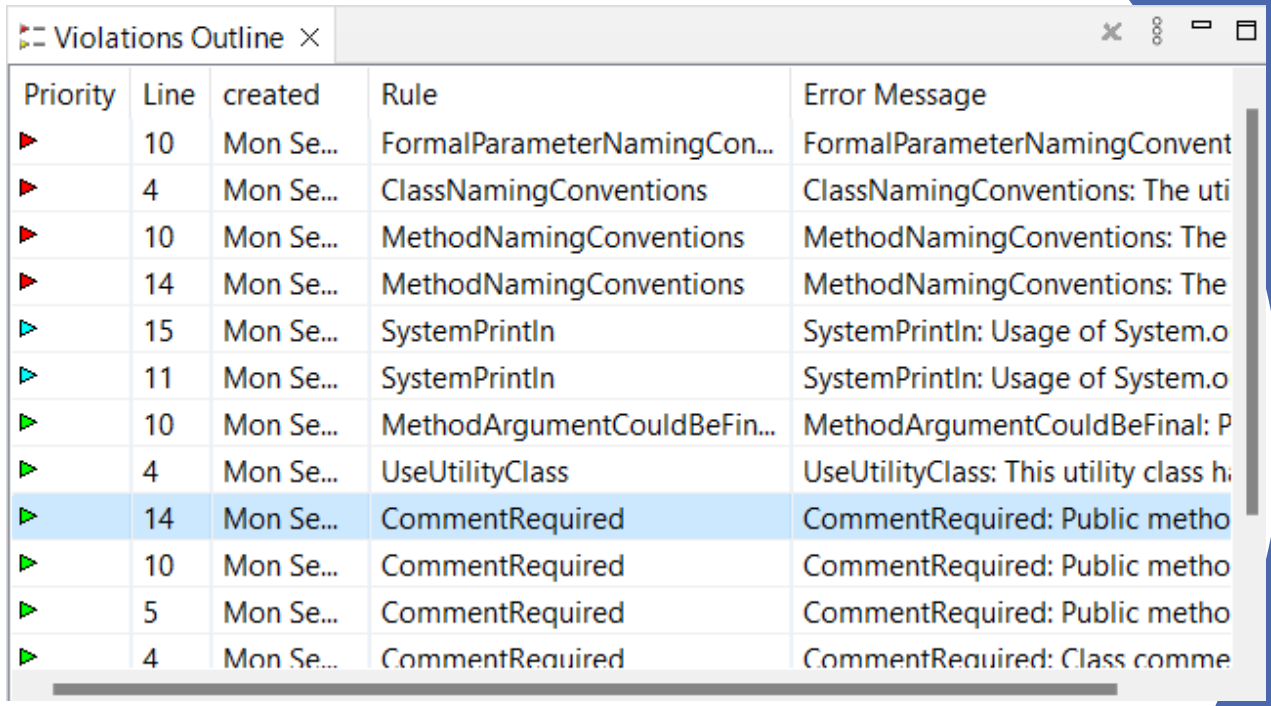
Now you can see a bug mark on the source code, double click the bug mark, Bug Explorer will be shown ( You can also navigate to **Window**->**Show View**->**Other**->**PMD**->**Violations Overview**).

```java
package demo1;


//PMD Eclipse Tutorial
public class demo1 {
public static void main(String args[]) {
demo1.CALL_METHOD("hello");
demo1.CallHello();
}


public static void CALL_METHOD(String
INPUT_PARAMETER) {
System.out.println(INPUT_PARAMETER);
}


public static void CallHello() {
System.out.println("Hello PMD World!");
}
}
```



# PMD rules along with Java example code that violates these rules



**Avoid Unused Local Variables Rule**

This rule detects unused local variables. Unused variables clutter your code and can lead to confusion.

```
public class UnusedVariableExample {
    public static void main(String[] args) {
        int x = 10;  // Unused variable
        int y = 20;  // Unused variable
        System.out.println("Hello, World!");
    }
}
```

**Avoid Unused Private Fields Rule**

This rule detects unused private fields in a class.

```
public class UnusedFieldExample {
    private int age; // Unused private field
    private String name; // Unused private field

    public UnusedFieldExample(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

**Cyclomatic Complexity Rule**

This rule detects methods with high cyclomatic complexity, which can be hard to understand and maintain.

```
public class CyclomaticComplexityExample {
    public void complexMethod() {  // High cyclomatic
complexity
        int x = 10;
        int y = 20;
        if (x > 5) {
            System.out.println("X is greater than 5");
            if (y < 30) {
                System.out.println("Y is less than 30");
            }
        } else {
            System.out.println("X is not greater than 5");
        }
    }
}
```

**Avoid Using Hardcoded Literals Rule**

This rule detects the use of hardcoded literals in code, which can make the code less maintainable.

```
public class HardcodedLiteralExample {
    public static void main(String[] args) {
        double totalPrice = 100.0;  // Hardcoded literal
        double discount = 10.0;    // Hardcoded literal
        double discountedPrice = totalPrice - discount;
        System.out.println("Discounted Price: " +
discountedPrice);
    }
}
```

**Avoid Empty Catch Blocks Rule**

This rule detects empty catch blocks, which can hide exceptions and make debugging difficult.

```
public class EmptyCatchBlockExample {
    public static void main(String[] args) {
        try {
            // Some code that may throw an exception
        } catch (Exception e) {
            // Empty catch block
        }
    }
}
```

**UnnecessaryConstructorRule**

This rule detects unnecessary constructors in a class.

```
public class UnnecessaryConstructorExample {
    public UnnecessaryConstructorExample() {
        // Unnecessary constructor
    }

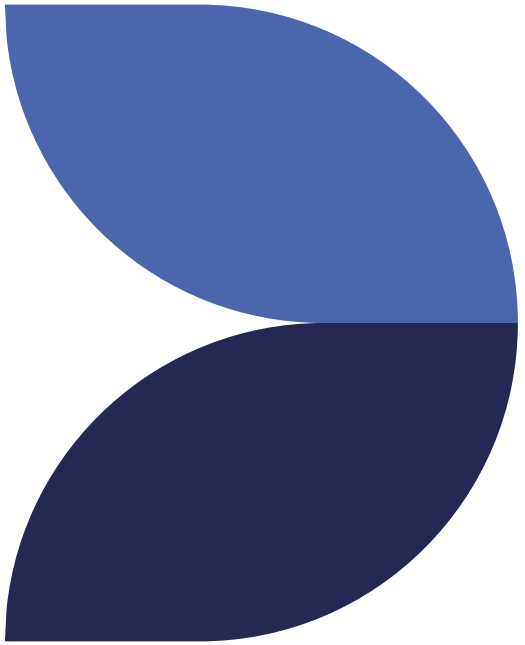
    public void doSomething() {
        // Some code here
    }
}
```

**Avoid Using Empty Statements Rule**

This rule detects empty statements, which serve no purpose and can lead to confusion.

```
public class EmptyStatementExample {
    public static void main(String[] args) {
        int x = 10;
        ; // Empty statement
        int y = 20;
    }
}
```

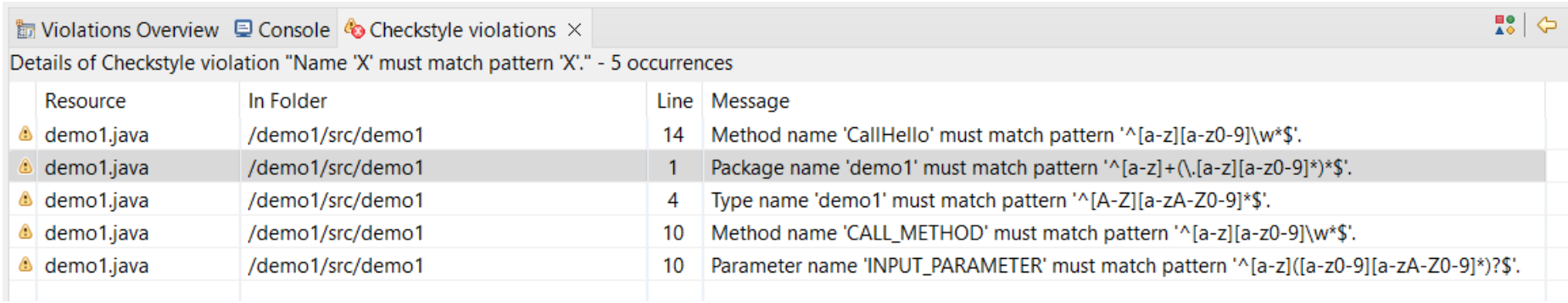
# CHECKSTYLE



It has a wideranging ruleset, ranging from details such as whitespace and formatting up to higherlevel issues relating to class design and complexity metrics.

Some of Checkstyle’s rules (such as CyclomaticComplexity) are also provided by PMD, so if you’re planning to use both tools, it’s best to tune your rulesets to avoid overlap. It’s also worth noting that, just like PMD, Checkstyle can be quite noisy if you enable all the rules. It’s a good idea to disable rules that aren’t relevant to your codebase or coding style. This can be done with an XML file and/or annotations, just like PMD.





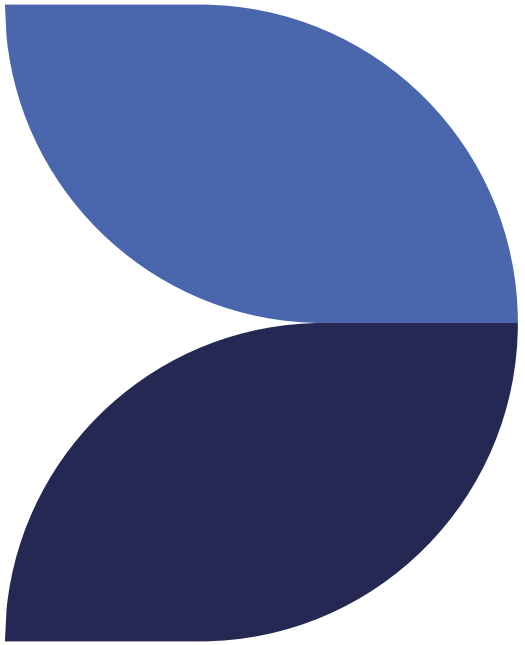

```
package demo1;


//PMD Eclipse Tutorial
public class demo1 {
public static void main(String args[]) {
demo1.CALL_METHOD("hello");
demo1.CallHello();
}


public static void CALL_METHOD(String INPUT_PARAMETER)
{
System.out.println(INPUT_PARAMETER);
}


public static void CallHello() {
System.out.println("Hello PMD World!");
}
}
```

# Continuous inspection using Jenkins



so far we've only looked at running the tools from inside a developer's IDE. This is a really useful way to inspect a project, but it has a couple of drawbacks:  Results are only visible to that developer. It's difficult to share this information with the rest of the team.  The process is dependent on developers remembering to run the tools regularly

Whenever a developer checks a new change into version control, a build is automatically triggered on the build server. The build server checks that the software builds, and it performs other tasks such as running static analysis tools or automated tests. If there's a problem with the build, such as if the change caused the code to stop compiling or introduced a new FindBugs warning, the build server will notify the developer who made the commit. With this quick feedback, the developer should be able to remedy their mistake easily. The build server also provides an online dashboard that team members can use to check the state of the software at any given time. This way, information about the software's quality is available to everybody, and nobody is left out of the loop. Using a build server in this way is sometimes called continuous inspection

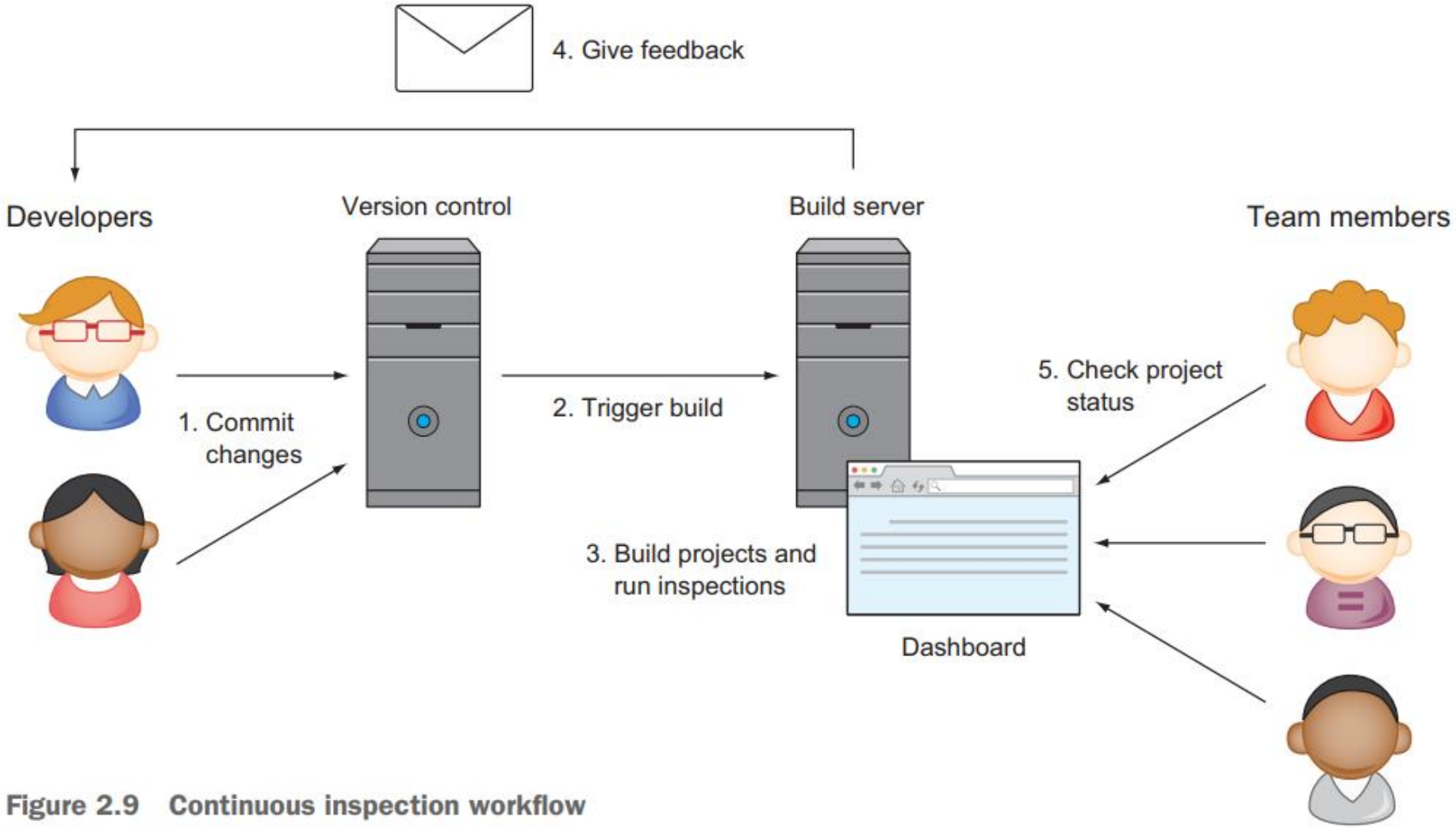



**Figure 2.9 Continuous inspection workflow**

# Installing and setting up Jenkins



## Jenkins is a popular tool for performing continuous integration of software projects

Jenkins needs a machine to run on. When you set Jenkins up for production use, it will need to be on a server that’s accessible to your whole team, but for now it’s probably easiest to install it on your local machine so you can play around with it. Jenkins is very easy to install, with native packages available for Windows, OS X, and various flavors of UNIX. It comes with an embedded web server, and it can automatically install most of the tools it needs, so the only thing you need to prepare in advance is a JDK.



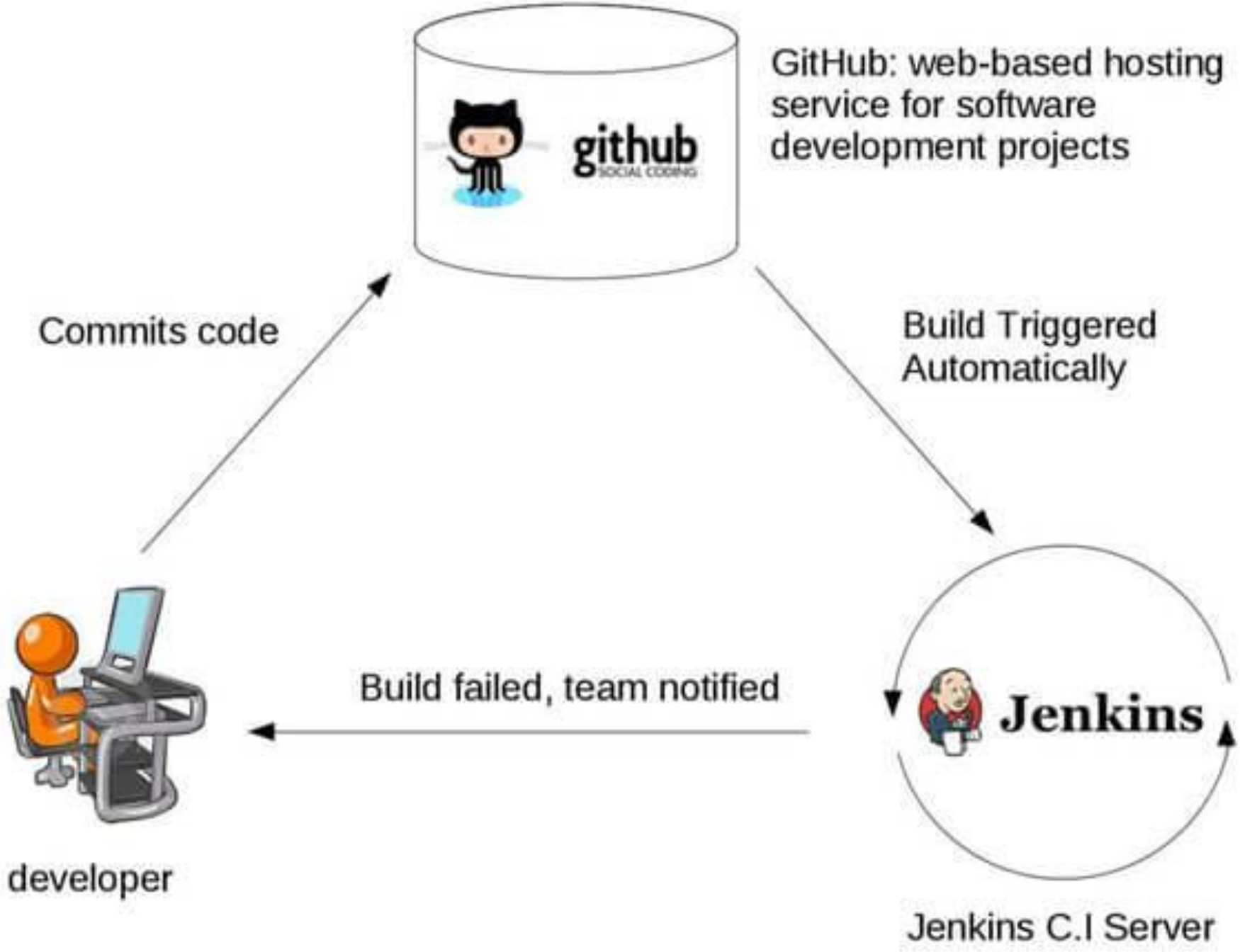
## What is Continuous Integration?

**Continuous Integration** is a process of integrating code changes from multiple developers in a single project many times. The software is tested immediately after a code commit. With each code commit, code is built and tested. If the test is passed, the build is tested for deployment. If the deployment is successful, the code is pushed to production.

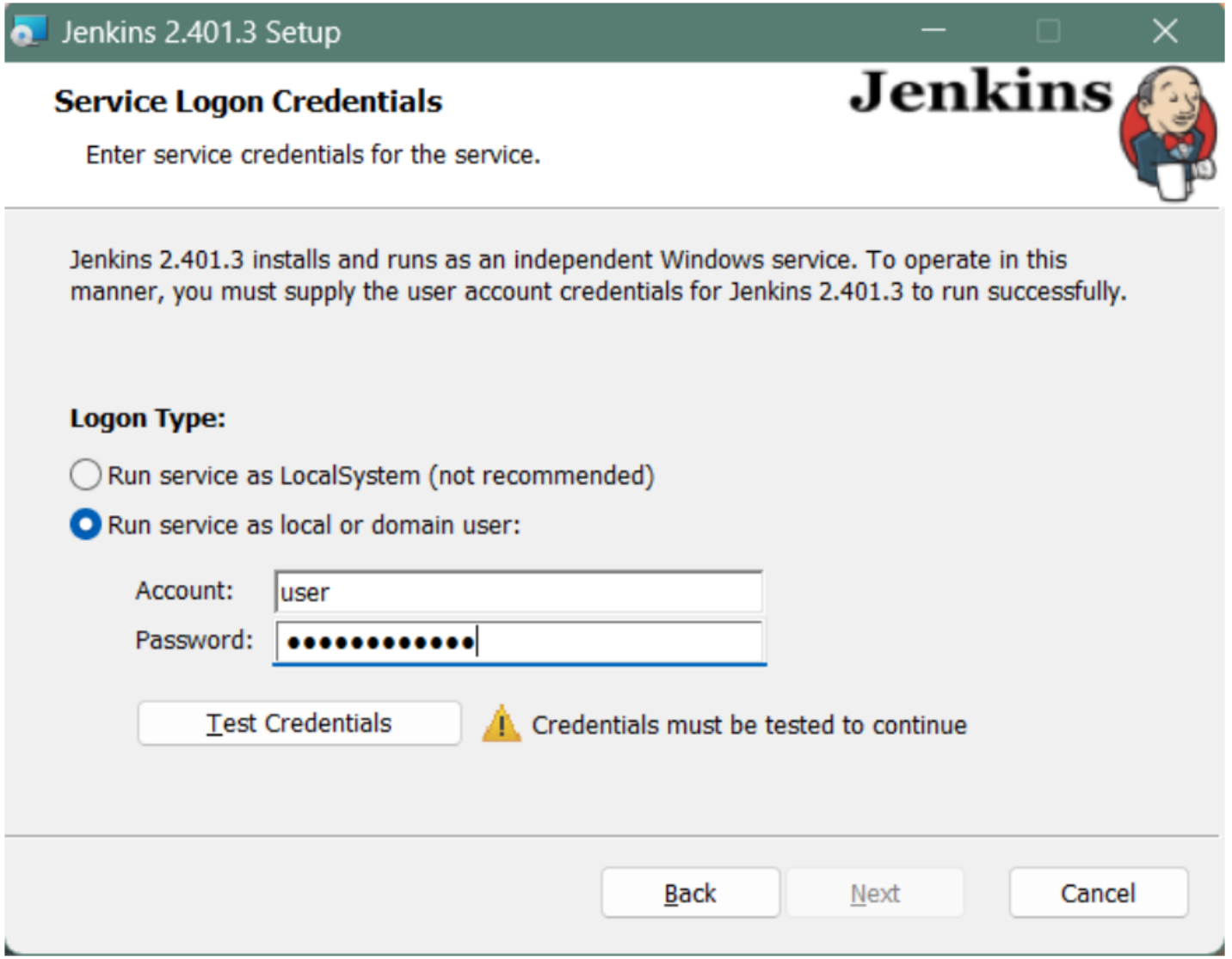
This commit, build, test, and deploy is a continuous process and hence the name continuous integration/deployment.

**Let us imagine, that there are around 10 developers who are working on a shared repository. Some developer completes their task in 25 days while others take 30 days to complete.**

| Before Jenkins | After Jenkins |
|---|---|
| Once all Developers had completed their assigned coding tasks, they used to commit their code all at same time. Later, Build is tested and deployed.<br>Code commit built, and test cycle was very infrequent, and a single build was done after many days. | The code is built and test as soon as Developer commits code.<br>Jenkin will build and test code many times during the day<br>If the build is successful, then Jenkins will deploy the source into the test server and notifies the deployment team.<br>If the build fails, then Jenkins will notify the errors to the developer team. |
| Since the code was built all at once, some developers would need to wait until other developers finish coding to check their build | The code is built immediately after any of the Developer commits. |
| It is not an easy task to isolate, detect, and fix errors for multiple commits. | Since the code is built after each commit of a single developer, it's easy to detect whose code caused the built to fail |
| Code build and test process are entirely manual, so there are a lot of chances for failure. | Automated build and test process saving timing and reducing defects. |
| The code is deployed once all the errors are fixed and tested. | The code is deployed after every successful build and test. |
| Development Cycle is slow | The development cycle is fast. New features are more readily available to users. Increases profits. |

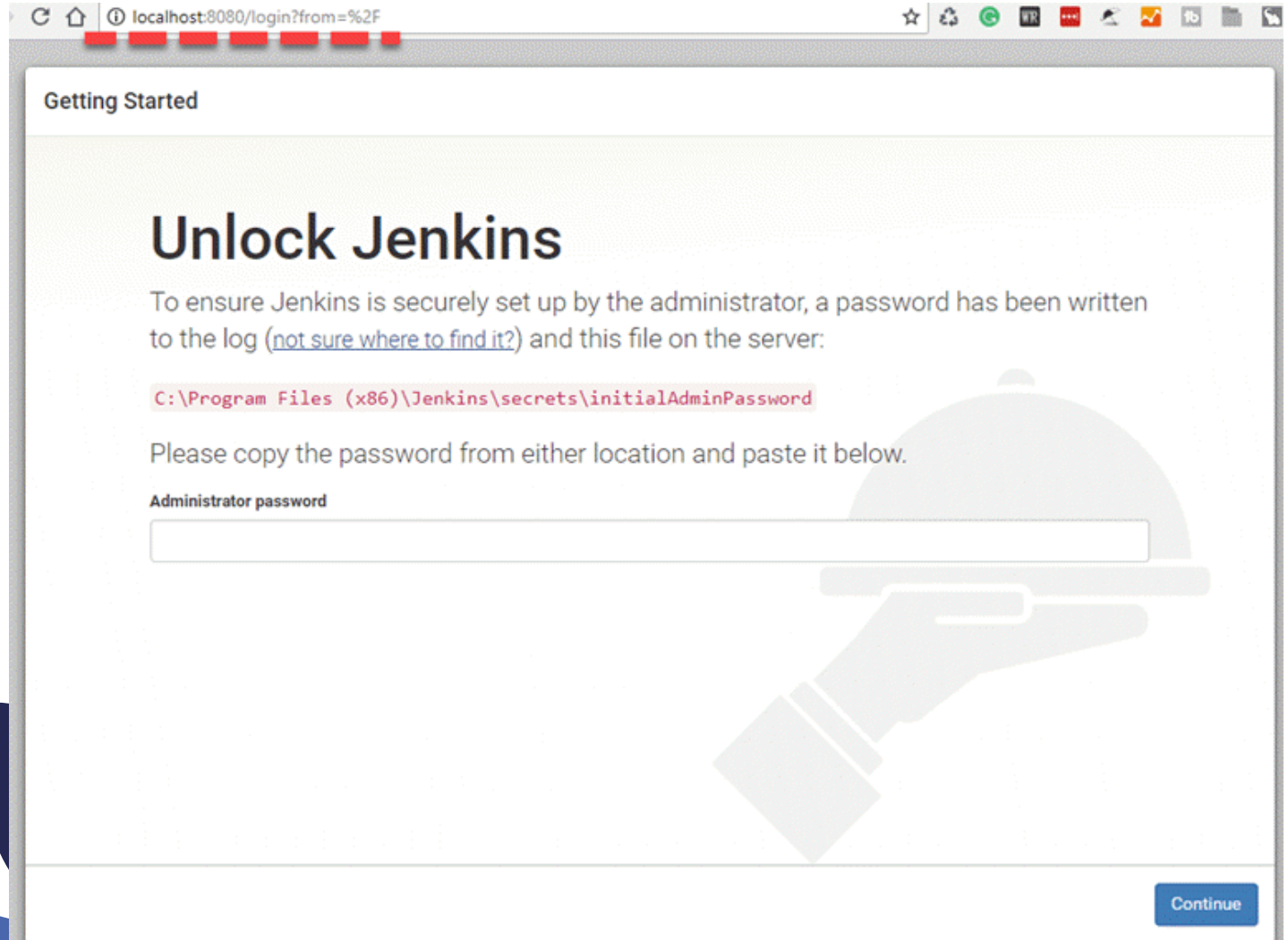

GitHub: web-based hosting service for software development projects
Commits code
Build Triggered Automatically
Build failed, team notified
developer
Jenkins C.I Server

download either an LTS release or a weekly release of the Windows installer.
https://www.jenkins.io/download/#downloading-jenkins

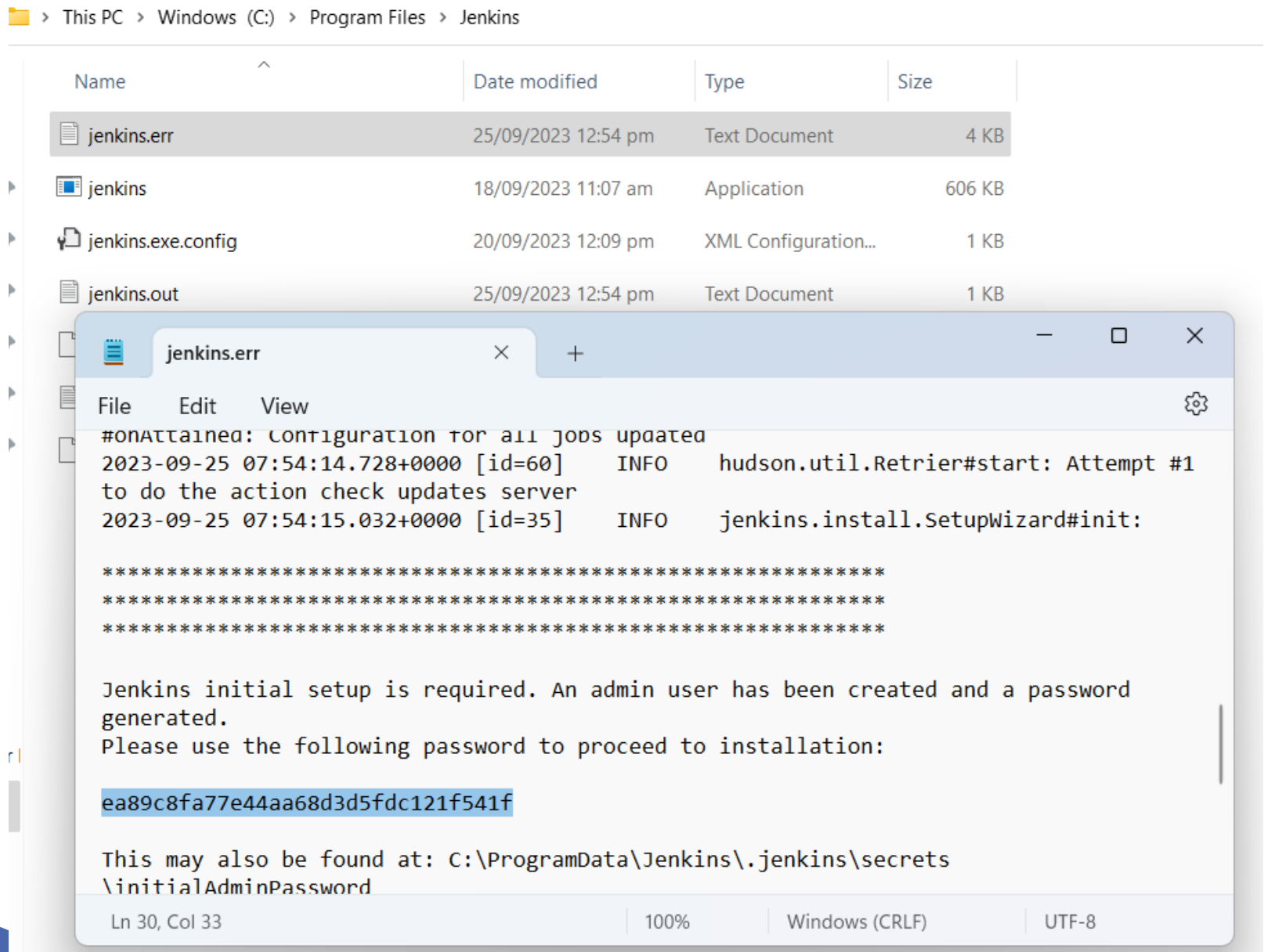


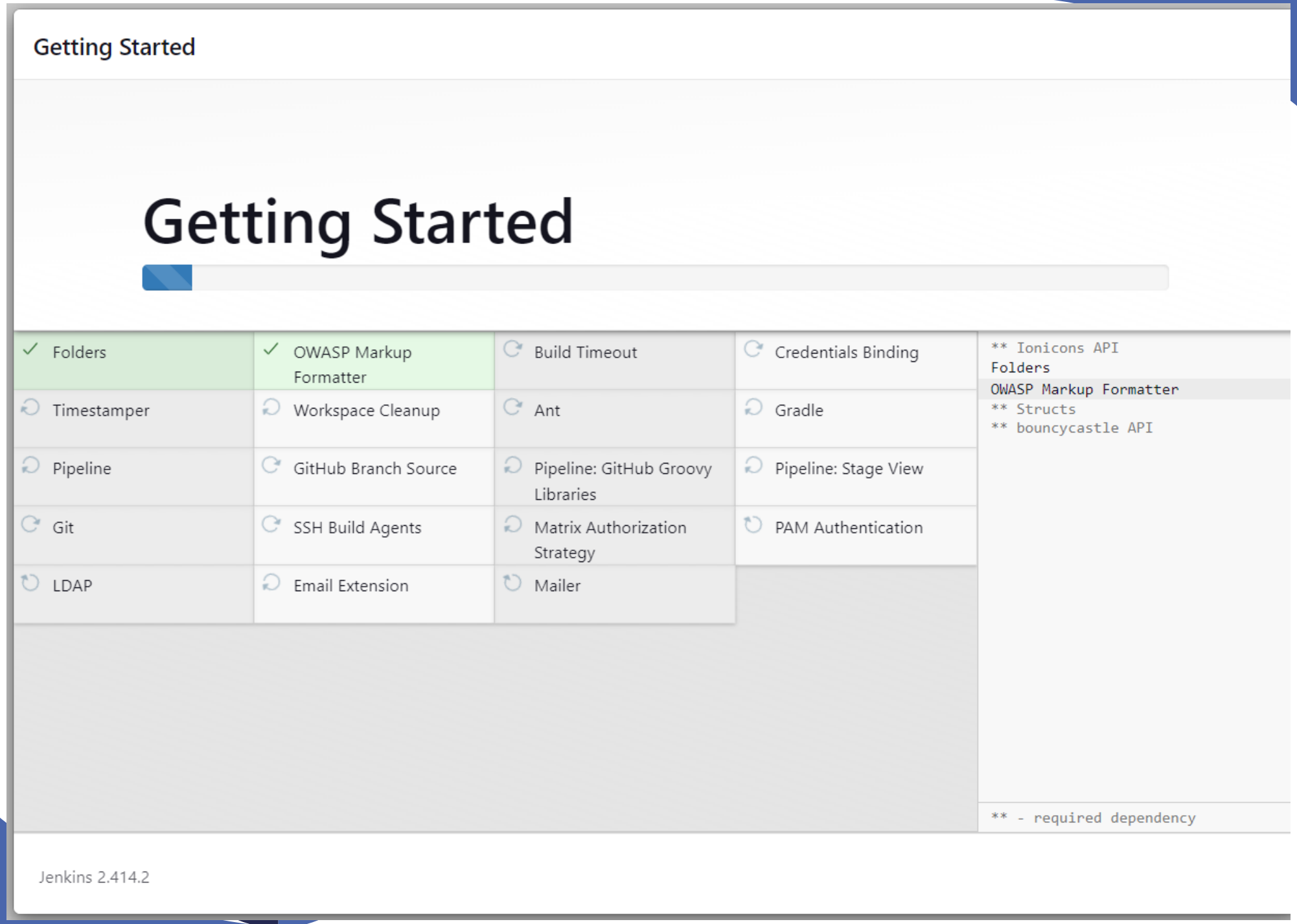
- After completing the Jenkins installation process, a browser tab will pop-up asking for the initial Administrator password.
- To access Jenkins, you need to go to browse the following path in your web browser.

**http://localhost:8080**



Pw:
ea89c8fa77e44aa68d3d5fdc121f541f




Getting Started
Getting Started
Folders
OWASP Markup Formatter
Build Timeout
Credentials Binding
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline: Stage View
Git
SSH Build Agents
Matrix Authorization Strategy
PAM Authentication
LDAP
Email Extension
Mailer
** Ionicons API
Folders
OWASP Markup Formatter
** Structs
** bouncycastle API
** - required dependency
Jenkins 2.414.2

# Integrating Jenkins with GitHub



Install git
https://git-scm.com/download/win

•First create a new job in Jenkins, open the Jenkins Dashboard and click on "create new jobs".

•Now enter the item name and select the job type. For example, item name is **javaTpoint**" and job type is "**Freestyle project**".
•Click on **OK**.

•Once you click OK, the page will be redirected to its project configuration. Enter the project information:

•Now, under the "Source Code Management" you will see the Git option, if your **Git** plugin has been installed in Jenkins:

•Enter the Git repository URL on the "Repository URL" option to pull the code from GitHub.

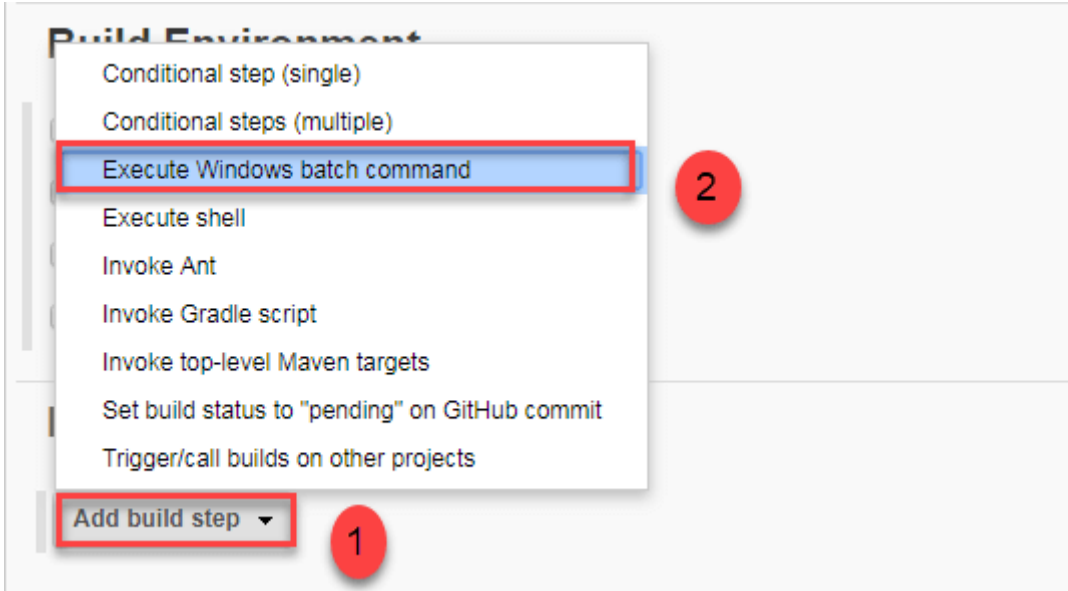
https://github.com/kriru/firstJava.git

**Tweak the settings**

Now that you have provided all the details, it's time to build the code. Tweak the settings under the **build** section to build the code at the time you want. You can even schedule the build to happen periodically, at set times.

Under **build**,

1. Click on "**Add build step**"
2. Click on "**Execute Windows batch command**" and add the commands you want to execute during the build process.

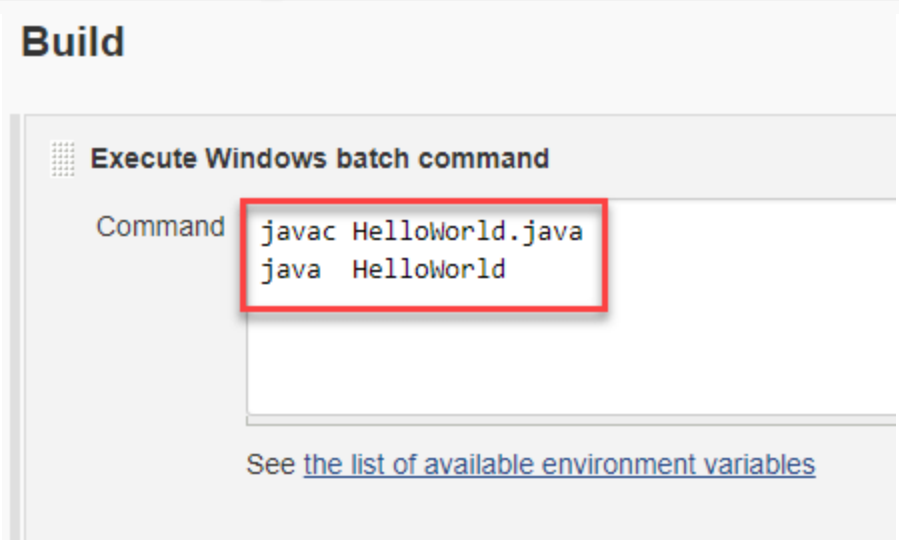



Here, I have added the java commands to compile the java code.

I have added the following windows commands:

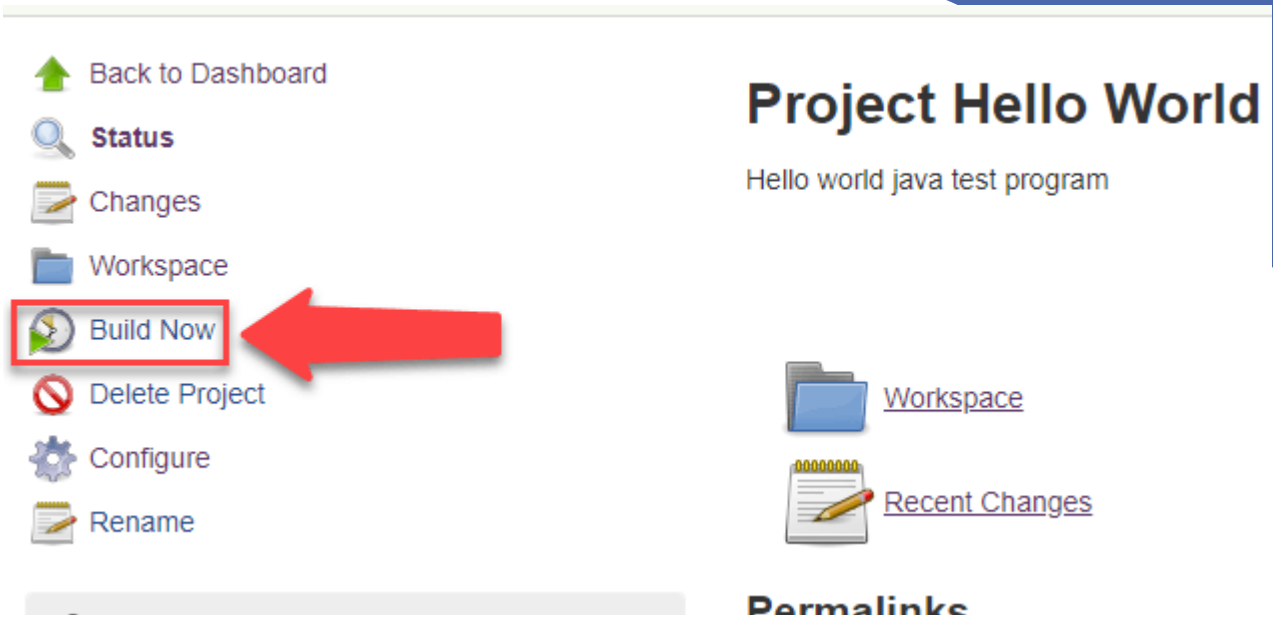
javac HelloWorld.java

java HelloWorld

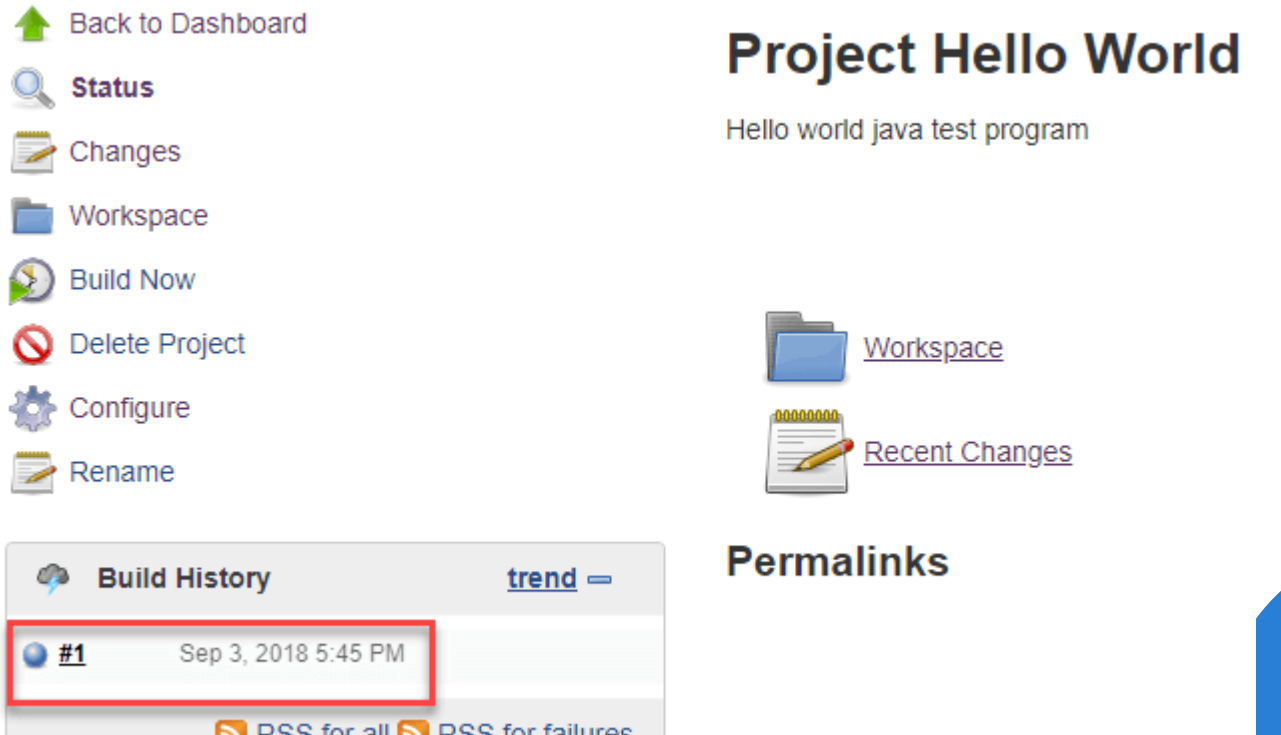


**Save the project**
When you have entered all the data,
1.Click **Apply**
**2.Save** the project.

**Step 8) Build Source code**
Now, in the main screen, Click the **Build Now** button on the left-hand side to build the source code.




**Step 9) Check the status**
After clicking on **Build now,** you can see the status of the build you run under **Build History**.



**See the console output**

Click on the **build number** and then Click on **console output** to see the status of the build you run.




It should show you a success message,

In sum, we have executed a HelloWorld program hosted on GitHub. Jenkin pulls the code from the remote epository and builds continuously at a frequency you define.

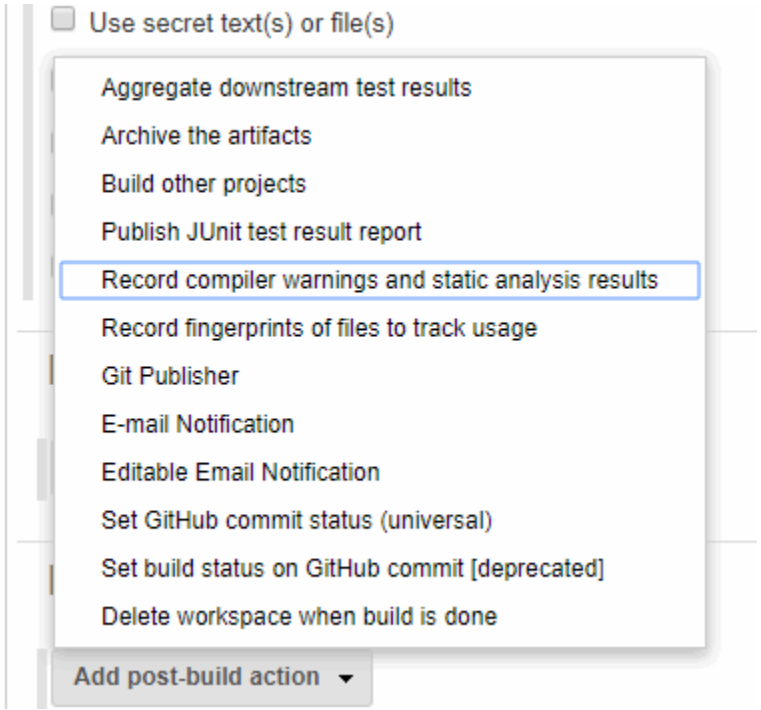
# Using Jenkins to build and inspect code

**Jenkins - Code Analysis**
The Jenkins Next Generation plugin helps in analyzing code, collects all warnings, and helps in visualizing the records through pie charts and graphs.

•Install Warnings Next Generation plugin from Manage Jenkins → Manage Plugins



task configuration page will open. Open the list "Add post-build action" at the bottom of this page in the item "Post-build Actions". In the list, select "Record compiler warnings and static analysis results":



Jenkins to inspect code using FindBugs, PMD, and Checkstyle

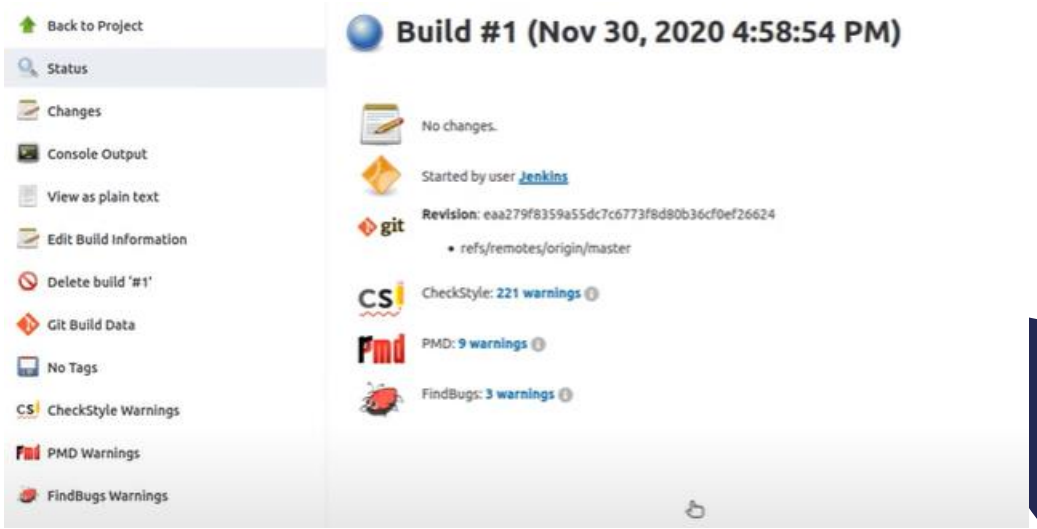
Add new itrem just as before
Build-> invoke top level maven targets

Goals-> clean compile findbugs:findbugs pmd:pmd checkstyle:checkstyle

Click apply

Add post build->record compile warnings and static analysis results

Build now



# Mandatory readings

- Chapter 2, Chris Birchall - Re-Engineering Legacy Software-Manning Publications (2016)



# Thank you